

WEEK 7 — CLASS REFERENCE
DOCUMENT

RAG Pipeline

Retrieval-Augmented Generation — A Complete Guide

This document covers everything discussed in Week 7 — from foundational concepts to the practical pipeline, failure diagnosis, and the connection between the chatbot you already built in Week 6 and the RAG system you are building now.

Section	Topic
1	From chatbot to RAG chatbot — what changes and why
2	What is RAG — definition, analogy, and the problem it solves
3	Key concepts — embeddings, semantic search, cosine similarity
4	The RAG pipeline — Phase 1: Ingestion
5	The RAG pipeline — Phase 2: Query
6	The full pipeline in code
7	Chunking strategy — the decisions that matter
8	Vector databases — what they are and how to choose
9	Failure diagnosis — the 5 modes and how to fix them
10	System prompt vs SKILL.md vs RAG — how they all fit together
11	Key takeaways and what comes next

■ This is the hardest week in the programme — not because the concept is complicated, but because there are many moving parts. When something breaks, it is not always obvious which part broke. This document is your map.

SECTION 1 — From Chatbot to RAG Chatbot

From Chatbot to RAG Chatbot — What Changes

In Week 6 you built a chatbot. It had a system prompt that controlled its persona, constraints, and format. Some of you also used a SKILL.md file — a document the agent reads on demand to learn how to perform a specific task. That chatbot works well. But it has a ceiling: it only knows what the model was trained on.

Now you are converting it to a RAG chatbot. Here is exactly what changes:

What	Regular Chatbot	RAG Chatbot
Knowledge source	Model's training data only	Your documents + model training
System prompt	Stays — controls behaviour	Stays — still controls behaviour
SKILL.md	Read on demand for task guidance	Can still exist alongside RAG
User question flow	Question → Model → Answer	Question → Retrieve → Augment → Model → Answer
Private info	Model has never seen it	Fetches fresh from your vector DB
Post-cutoff info	Model doesn't know it	Retrieved from your updated documents
Hallucination risk	Higher — answers from memory	Lower — grounded in retrieved context

The key insight

Your system prompt doesn't go away. It still sits at the top of every conversation, controlling how the model behaves. RAG adds a layer underneath it — a retrieval system that feeds the model relevant information before it generates its response. Week 6 taught you to control the model's behaviour. Week 7 teaches you to control the model's knowledge.

What stays the same:

- Your system prompt — persona, constraints, format instructions, examples
- The conversation loop — user sends a message, model replies
- The API call structure — system, user, assistant message roles

What is new:

- An ingestion pipeline — you clean, chunk, embed, and store your documents once
- A retrieval step — every query now searches a vector database before reaching the model

- An augmentation step — retrieved chunks are injected into the prompt as context
- A vector database — where your document embeddings live

SECTION 2 — What is RAG

What is RAG?

RAG stands for **Retrieval-Augmented Generation**.

Letter	Word	What it means
R	Retrieval	Find relevant information from your documents before answering
A	Augmented	Add that retrieved information to the model's prompt as context
G	Generation	The model generates an answer using the retrieved context — not just memory

One-line definition

RAG = find relevant information first, then ask the model to answer using it — instead of asking the model to answer from memory alone.

The Two Problems RAG Solves

Every language model has two fundamental limitations. RAG is the direct fix for both.

Problem 1 — Knowledge cutoff

Every model was trained on data up to a certain date. It knows nothing after that — not your company's latest policy update, last week's product launch, or this year's regulations. RAG fix: fetch current information at runtime from documents you control. The model does not need to have memorised it.

Problem 2 — Private information

The model has never seen your internal documents, your customer database, your student handbook, your codebase. It cannot answer questions about things it has never read. RAG fix: retrieve from your private documents. The model reads them fresh for every query.

The analogy:

Without RAG: you ask a brilliant person a question — but they have been on a desert island for two years with no news and no access to your company files. With RAG: same brilliant person, but right before they answer, you hand them the exact documents they need to read first. They are still the one generating the answer — they just have the right information in front of them.

SECTION 3 — Embeddings, Semantic Search & Cosine Similarity

Embeddings — How Text Becomes Searchable Numbers

The heart of RAG is the ability to search documents by *meaning*, not just by exact keywords. Normal search (Ctrl+F) finds exact word matches. If a document says 'password recovery steps' and you search 'how do I reset my password' — keyword search finds nothing. The words do not match.

Embeddings fix this. An embedding converts text into a list of numbers — typically hundreds or thousands of them — that captures the **meaning** of that text.

```
"how do I reset my password" → [0.23, -0.81, 0.44, 0.12, ...] (768 numbers)
"password recovery steps" → [0.21, -0.79, 0.46, 0.14, ...] (768 numbers)
"I love eating jollof rice" → [-0.55, 0.33, -0.12, 0.88, ...] (768 numbers)
```

Notice the first two lists look very similar. The third looks very different. That is the point — **similar meanings produce similar numbers**. This is what enables semantic search.

The Geometry of Meaning

Think of each piece of text as a point in space. Texts with similar meanings end up **close together**. Texts with different meanings end up **far apart**. When you search, you embed your query — place it as a point in that same space — then find the document chunks that are closest to it.

■ Close in meaning-space

- reset password ↔ password recovery
- refund policy ↔ money back process
- how to enrol ↔ registration steps

■ Far in meaning-space

- reset password ↔ jollof rice recipe
- refund policy ↔ football match results
- how to enrol ↔ car engine repair

Cosine Similarity — How Closeness is Measured

The standard way to measure how close two embeddings are is called cosine similarity. It gives a score between -1 and 1. In practice, you will work with scores between 0 and 1.

Score range	What it means	What to do
0.9 – 1.0	Near-identical meaning	Always retrieve
0.75 – 0.9	Strongly related	Retrieve

0.5 – 0.75	Somewhat related	Retrieve with caution
Below 0.5	Probably noise	Do not retrieve — raise your threshold

This is the score you see when you print retrieved chunks. Low scores (0.3, 0.4) mean nothing genuinely relevant exists in your database for that query. The system is returning noise.

Important

You do not need to understand the mathematics behind embeddings to use them correctly. You need to understand what they do: convert text to numbers that capture meaning, so you can search by meaning instead of exact words.



SECTION 4 — The RAG Pipeline: Phase 1 — Ingestion

Phase 1 — The Ingestion Pipeline (Build Once)

Think of this as **building the library**. You run this once when setting up your system, and again whenever your documents change. Every subsequent query will search the library you built here.

01	02	03	04	05
■ Raw Documents	■ Clean	>■ Chunk	■ Embed	■■ Store
PDFs, Word docs, text files, web pages	Remove headers, footers, page numbers, duplicates	Cut into 256–512 token pieces with overlap	Convert each chunk to a list of numbers	Save text + embedding in vector database

Step 02 — Cleaning (Most Critical)

This is where more RAG systems fail than anywhere else. If your documents contain headers, footers, page numbers, or repeated boilerplate mixed into the body text, the embedding model embeds the garbage alongside the content. Retrieval becomes unreliable. Fix this before touching anything else.

What to clean:

- Page headers and footers ('Company Name | Confidential | Page 3 of 12')
- Page numbers that appear as standalone lines
- Repeated boilerplate paragraphs (copyright notices, disclaimer text)
- Navigation menus and sidebar text if scraping from web
- Table of contents entries mixed into body text
- Watermark text extracted by PDF parsers

Step 03 — Chunking Strategy

You cannot embed an entire document as one unit. Two reasons: context window limits mean you cannot fit whole documents into the model's prompt, and a 50-page document is 95% irrelevant to any given question. You want to retrieve the two paragraphs that actually matter.

■ ■ Too small (under 100 tokens)

- Loses surrounding context
- "This applies to the above policy" — meaningless alone
- Embedding averages over too little signal
- Retrieval becomes imprecise

■ ■ Too large (over 1000 tokens)

- Contains too much irrelevant content
- Relevant sentence buried in noise
- Chunk embedding is diluted
- Context window fills up fast

Sweet spot

256–512 tokens per chunk — roughly 2–4 paragraphs. This is a starting point, not a rule. Dense technical text needs smaller chunks. Narrative text can handle larger ones. Test with your actual documents and measure retrieval quality.

Overlap — the trick most beginners skip:

When you cut a document into chunks, important information often sits right at the boundary between two chunks. The fix is overlap — each chunk shares some text with the next one. Typically 10–20% overlap. It means you store slightly more data, but boundary sentences always appear whole in at least one chunk.

```
Chunk 1: [paragraph A] [paragraph B] [first half of C] Chunk 2: [paragraph B]
[paragraph C] [first half of D] Chunk 3: [paragraph C] [paragraph D] [paragraph E]
```


SECTION 5 — The RAG Pipeline: Phase 2 — Query

Phase 2 — The Query Pipeline (Runs Every Time)

Think of this as **using the library**. Every time a user asks a question, this entire sequence runs. Speed matters here — every step adds latency.

01	02	03	04	05	06
■ User Question	■ Embed Question	■ Search Vector DB	■ Retrieve Chunks	■ Augment Prompt	■ Generate Answer
"What is the refund policy?"	Same embedding model as ingestion	Find top-k most similar chunks	Pull the 3–5 most relevant pieces	Add chunks as context to prompt	Model reads context and responds

Why each step cannot be skipped:

Step	Why it exists
Embed question	Must use the same model as ingestion — different models produce incompatible number spaces
Search vector DB	Without similarity search you cannot find semantically relevant chunks
Retrieve chunks	You cannot pass 500 documents to the model — you need to filter to what's relevant
Augment prompt	The model cannot query the database itself — you must hand it the information explicitly
Generate answer	The pipeline retrieves — it does not answer. The model still does the generation.

Critical rule

Always use the same embedding model for ingestion and querying. If you embed your documents with model A and embed the query with model B, the numbers live in completely different spaces. Similarity search will return garbage — even if both models are good individually.

What top-k means:

Top-k is the number of chunks retrieved before generation. Typical values are 3–5. More chunks = more context = potentially better answers, but also higher latency, higher token cost, and more noise if your

retrieval is not precise.

top-k value	Pros	Cons
1–2	Fast, focused, low cost	May miss relevant information
3–5	Good balance for most use cases	Moderate latency and cost
10+	High recall — less likely to miss anything	More noise, higher latency, more tokens

Start with top-k=3 or 5. Adjust based on your eval results in Week 8.

SECTION 6 — The Full Pipeline in Code

A Complete RAG Query — Python

This is what a complete RAG query looks like in code. Every RAG system you build is a variation of this pattern.

```
# 1. User asks a question question = "What is the refund policy for digital
products?" # 2. Embed the question using the SAME model used during ingestion
question_embedding = embedding_model.embed(question) # 3. Search the vector DB for
similar chunks results = vector_db.search(question_embedding, top_k=5) # 4. Print
chunks for diagnosis (always do this during development) for r in results:
print(f"Score: {r.score:.3f} | {r.text[:150]}") # 5. Build context from retrieved
chunks context = "\n\n".join([r.text for r in results]) # 6. Augment the prompt –
system prompt still controls behaviour system_prompt = """ You are Ama, a support
agent for Codetrain Africa. Answer only using the information provided in the
context below. If the answer is not in the context, say so clearly. Never make up
information. """ user_prompt = f""" Context: {context} Question: {question} """ #
7. Call the model response = client.messages.create( model="claude-sonnet-4-6",
system=system_prompt, messages=[{"role": "user", "content": user_prompt}] )
```

Notice

The system prompt is still there — it controls how the model behaves. The retrieved context goes into the user message as information for the model to read. The model's job is only generation — it never touches the vector database directly.

The Retrieval Diagnostic — Run This Every Time Results Look Wrong

Before you blame the model, always check what was actually retrieved. Add this print block temporarily and read the output carefully.

```
print("=== Retrieved chunks ===") for i, chunk in enumerate(retrieved_chunks):
print(f"\nChunk {i+1}") print(f"Score: {chunk.score:.3f}") print(f"Text:
{chunk.text[:200]}") print("-" * 40) # Then ask yourself: # Are these chunks
relevant to the query? # If YES → retrieval works. The generation prompt is the
problem. # If NO → retrieval is broken. Why were these chunks returned?
```

SECTION 7 — Vector Databases

Vector Databases — What They Are

A vector database stores each chunk's text alongside its embedding (the list of numbers). When you give it a query embedding, it finds the stored embeddings that are most similar and returns the corresponding chunks, ranked by similarity score.

The vector database does not understand language at all. It does pure geometry — finding nearby points in high-dimensional space very efficiently.

Common options:

Database	Type	Best for
Chroma	Local / in-process	Learning, prototyping, small projects
FAISS	Local library	Fast local search, research projects
Pinecone	Managed cloud service	Production systems, no infrastructure management
Weaviate	Self-hosted or cloud	Production with rich filtering needs
pgvector	PostgreSQL extension	Teams already using PostgreSQL
Qdrant	Self-hosted or cloud	High performance production systems

For this week

Use Chroma. It runs locally, requires no account, and works in-process with Python. Once you understand the pipeline, switching to a production database like Pinecone is mostly a configuration change — the pipeline logic stays the same.

How a Vector DB Query Works

```
import chromadb # Create client and collection client = chromadb.Client()
collection = client.create_collection("codetrain_docs") # Add documents (ingestion)
collection.add( documents=[chunk.text for chunk in chunks],
embeddings=[chunk.embedding for chunk in chunks], ids=[f"chunk_{i}" for i in
range(len(chunks))] ) # Query (at runtime) results = collection.query(
query_embeddings=[question_embedding], n_results=5 ) # results['documents']
contains the retrieved chunk texts # results['distances'] contains the similarity
scores
```

SECTION 8 — Failure Diagnosis — The 5 Modes

The 5 Places a RAG System Breaks

When a RAG system gives a bad answer, diagnose in this order — top to bottom. **Do not touch the generation prompt until you have confirmed the retrieved chunks are actually relevant.** Beginners almost always blame the model. The problem is almost always in the first two steps.

#	Failure mode	Root cause	Fix	How common
0 1	Dirty documents	Headers, footers, page numbers, duplicates corrupt embeddings	Clean documents before chunking. This is always Step 1.	Most common
0 2	Wrong chunk size	Too small loses context. Too large dilutes the signal.	Experiment with 256, 512, 1024 token sizes. Measure retrieval quality.	Very common
0 3	Wrong embedding model	General model can't handle specialist vocabulary	Use a domain-specific embedding model or fine-tune.	Common
0 4	Threshold too low	Returning chunks with scores of 0.3–0.4 when nothing relevant exists	Set a minimum score threshold. If nothing scores above it, say 'I don't know'.	Common
0 5	Bad generation prompt	Good chunks retrieved but model ignores them or misuses them	Explicitly instruct the model to use only the provided context.	Less common

The Diagnostic Workflow

Every time a user reports a bad answer, follow this exact sequence:

- **Step 1:** Print the retrieved chunks and their scores
- **Step 2:** Ask — are these chunks relevant to the question?
 - If YES → retrieval works. Move to Step 4.
 - If NO → retrieval is broken. Move to Step 3.
- **Step 3 (if retrieval broken):** Why were these chunks retrieved? What do they have in common with the query? Check for dirty documents first, then chunk size, then embedding model.
- **Step 4 (if retrieval works):** Look at your system prompt. Are you telling the model to use the context? Are you telling it to say 'I don't know' when the context doesn't contain the answer?
- **Step 5:** If everything looks correct, check the score threshold. If all scores are below 0.5 and you're still returning results, you're feeding the model noise.

The rule

Before you blame the model — look at what it was given. A model that hallucinates when given relevant context has a prompt problem. A model that hallucinates when given irrelevant context has a retrieval problem. These are very different issues with very different fixes.

SECTION 9 — System Prompt vs SKILL.md vs RAG — How They Fit Together

How System Prompt, SKILL.md, and RAG Work Together

These three things are not alternatives to each other. They operate at different layers of the same system. Understanding how they relate is key to building production AI systems.

Layer	What it is	When it runs	What it controls
System prompt	Instructions in the API call	Every single conversation	Persona, behaviour, constraints, format — how the model acts
SKILL.md	A file read from disk	On demand — when the agent needs to perform a specific task	How to do one specific task — operational knowledge for the agent
RAG	A retrieval pipeline + vector database	Every query — retrieval runs before generation	What the model knows — real-time access to your documents

How they layer in a real system:

```
# Layer 1 – System prompt (always present, controls behaviour) system_prompt = ""
You are Ama, Codetrain Africa support agent. Answer only using provided context.
Never make up information. "" # Layer 2 – SKILL.md (read on demand if agent needs
task guidance) if task_requires_skill: skill_content =
open('skills/support_responses.md').read() system_prompt += "\n" + skill_content #
Layer 3 – RAG (retrieval runs before every generation) retrieved_chunks =
vector_db.search(question_embedding, top_k=5) context = "\n".join([c.text for c in
retrieved_chunks]) # All three layers combine in the final prompt user_prompt =
f"Context:\n{context}\n\nQuestion: {question}"
```

The analogy:

The system prompt is the employee handbook — always with you, always in effect. The SKILL.md is the specialist manual — pulled off the shelf when you need to do a specific task correctly. RAG is the library — you search it fresh every time someone asks a question and hand the relevant pages to whoever is answering.

Key Questions Students Ask

Q: Do I still need a system prompt if I have RAG?

Yes. RAG gives the model information. The system prompt tells it how to behave with that information — whether to say 'I don't know' when context is missing, what tone to use, what it should never do.

Q: Is SKILL.md the same as a system prompt?

No. A system prompt is in the API call and runs every conversation. A SKILL.md is a file on disk, read on demand and injected into context only when the agent decides it's needed for a specific task. They are both instructions for the model, but they operate at different times and for different purposes.

Q: Can RAG replace the system prompt?

No. RAG provides knowledge. The system prompt provides behaviour. You need both. A model with perfect knowledge but no behavioural constraints will still answer in the wrong format, reveal things it shouldn't, or ignore its persona.

SECTION 10 — Key Takeaways & What Comes Next

Key Takeaways — Week 7

What RAG is	Retrieval-Augmented Generation = find relevant information first, then generate an answer using it as context — not from memory alone.
Why it exists	It solves two fundamental model limitations: knowledge cutoff dates and inability to access private documents.
Embeddings	Text converted to lists of numbers that capture meaning. Similar meanings → similar numbers → semantic search.
Ingestion pipeline	Clean → Chunk (256–512 tokens with overlap) → Embed (same model always) → Store in vector DB. Run once.
Query pipeline	Embed question → Search DB → Retrieve top-k → Augment prompt → Generate. Runs every query.
Most critical step	Cleaning. Dirty documents — headers, footers, duplicates — corrupt embeddings and break retrieval. Fix this first.
Diagnosis rule	When results are bad, print the retrieved chunks and their scores BEFORE blaming the model. It is almost never the model.
System prompt still matters	RAG gives the model knowledge. The system prompt gives the model behaviour. You need both.
One embedding model	Always use the same embedding model for ingestion and querying. Different models = incompatible number spaces = garbage retrieval.

What Comes Next — Week 8

Week 8 is the Month 2 Gate. You will need:

- A live RAG system responding to 3 unseen queries chosen by the instructor
- An eval dataset of 20 queries with a documented baseline score
- Reranking implemented — and the ability to explain the latency/quality trade-off for your specific use case
- Blog Post 2 live at a public URL

The question that separates serious students

"What are the three worst-performing queries in your eval set? Why do they fail?" A student who cannot name specific failing queries has not engaged with their eval dataset seriously. The eval is not a box to tick — it is how you know if your system works.

Next session covers: chunking experiments, reranking, hybrid search, and building your eval dataset.